SEARCHING AND SORTING

Closest Numbers

```
#include<stdio.h>
#include<stdlib.h>

void quickSort(int[], int, int);
int partition(int[], int, int);

int main(){
	int i,n;
	//input
	scanf("%d",&n);
	int *a=(int *)malloc(n*sizeof(int));

	for(i=0;i<n;i++)
		scanf("%d",&a[i]);
	//sorting
	quickSort(a,0,n-1);

	//finding smallest
	int min=a[1]-a[0];
	for(i=2;i<n;i++)
		if(a[i]-a[i-1]<min) min=a[i]-a[i-1];

	//printing all pairs
	for(i=1;i<n;i++)
		if(a[i]-a[i-1]==min) printf("%d %d ",a[i-1],a[i]);
	printf("\n");
```

```
return 0;
}

void quickSort(int a[], int l, int r)
{
   int j;
   if( l < r )
   {
        // divide and conquer
       j = partition( a, l, r);
      quickSort( a, l, j-1);
      quickSort( a, j+1, r);
   }
}

int partition(int a[], int l, int r) {
   int pivot, i, j, t;
   pivot = a[l];
   i = l; j = r+1;
   while( 1)
   {
        do ++i; while( a[i] <= pivot && i <= r );
        do --j; while( a[j] > pivot );
        /*do ++i; while( a[i] >= pivot && i <= r );
        do --j; while( a[j] < pivot );*/
        if( i >= j ) break;
        t = a[i]; a[i] = a[j]; a[j] = t;
   }
```

```
    t = a[l]; a[l] = a[j]; a[j] = t;
    return j;
}
```

Ice Cream Parlor

```
#include<stdio.h>
int main()
{
    int t,c,l,i,j,arr[20000];
    scanf("%d",&t);
    for( ; t>0 ; t--)
    {
        scanf("%d%d",&c,&l);
        for(i=0;i<l;i++)
        scanf("%d",&arr[i]);
        for(i=0;i<l-1;i++)
        for(j=i+1;j<l;j++)
        {
            if(arr[i]+arr[j]==c)
            printf("%d %d\n",i+1,j+1);
        }
    }
    return 0;
}
```

Find the Median

RECURSION AND BIT MANIPULATION

Maximizing XOR

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <assert.h>
int maxXor(int l, int r) {
    int max = 0,i,j;
    for(i=l;i<r;i++)
        for(j=i+1;j<=r;j++)
            max = max<(i^j)?i^j:max;
    return max;
}
int main() {
    int res;
    int _l;
    int _r;
    scanf("%d", &_l);
    scanf("%d", &_r);
    res = maxXor(_l, _r);
    printf("%d", res);
    return 0;
}
```

Sum vs XOR

```
#include <stdio.h>
int main(){
        long long int n,m=1;
        scanf("%lld",&n);
        while(n>0){
                if(n%2==0)m*=2;
                n/=2;
        }
        printf("%lld\n",m);
        return 0; }
```

Flipping Bits

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
int main() {
    int t;
    unsigned int n;
    scanf("%d", &t);
    while(t-- > 0) {
        scanf("%u", &n);
        printf("%u\n", ~n);
    }
    return 0;
}
```

GREEDY AND DYNAMIC PROGRAMMING

Mark and Toys

```
#include<stdio.h>
void quicksort(int x[100000],int first,int last){
    int pivot,j,temp,i;

     if(first<last){
         pivot=first;
         i=first;
         j=last;

         while(i<j){
             while(x[i]<=x[pivot]&&i<last)
                 i++;
             while(x[j]>x[pivot])
                 j--;
             if(i<j){
                 temp=x[i];
                  x[i]=x[j];
                  x[j]=temp;
             }
         }

         temp=x[pivot];
         x[pivot]=x[j];
         x[j]=temp;
         quicksort(x,first,j-1);
         quicksort(x,j+1,last);
```

```
    }}

int main()
{
int n,k,i,avail=0,count=0;
int cost[n];
scanf("%d",&n);
scanf("%d",&k);
for(i=0;i<n;i++)
scanf("%d",&cost[i]);
quicksort(cost,0,n-1);
while(avail<=k)
{
avail+=cost[count];
count++;
}
printf("%d\n",count-1);
return 0;    }
```

Greedy Florist

```
#include <stdio.h>
```

```
#include <stdlib.h>

int a[200],i,j,k,l,sum,n;

int com(const void *xx, const void *yy)
{
if(*(int *)xx > *(int*)yy) return -1;
return 1;
}

int main()
{

scanf("%d %d\n",&n,&k);
for(i=0;i<n;i++)
scanf("%d",a+i);

qsort(a,n,sizeof(a[0]),com); // sorts in descending order of price
sum = 0;
for(i=0;i<n;i++)
  sum+= a[i]*(1+i/k);
printf("%d\n",sum);
return 0;
}
```

STRINGS

Camelcase

```
#include <math.h>
#include <stdio.h>
```

```
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <limits.h>
#include <stdbool.h>

int main(){
    char* s = (char *)malloc(10240 * sizeof(char));
    scanf("%s",s);
    int count=0,i;
    for(i=0;i<strlen(s);i++)
    {
        if(s[i]>=65 && s[i]<=90)
            {
                count++;
            }
    }
    printf("%d\n",count+1);
    return 0;
}
```

Mars Exploration

```
#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```
#include <assert.h>
#include <limits.h>
#include <stdbool.h>
int main(){
    char* S = (char *)malloc(10240 * sizeof(char));
    scanf("%s",S);
    int i;
    int count=0;
    for(i=0;S[i]!='\0';i+=3){
        if(S[i]!='S'){
            count++;
        }
        if(S[i+1]!='O'){
            count++;
        }
        if(S[i+2]!='S'){
            count++;
        }
    }
    printf("%d",count);
    return 0;
}
```

Funny String

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
```

```
int main() {
    int T, N, funny, i, r;
    char S[10001];

    scanf("%d\n", &T);
    while (T--) {
        scanf("%s\n", S);
        N = strlen(S);
        funny = 1;
        for(i=1, r=N-2; i<N; i++, r--) {
            if (fabs(S[i]-S[i-1]) != fabs(S[r]-S[r+1])) {
                funny = 0;
                break;
            }
        }
        if (funny)
        {
         printf("funny");
                }
                else
                printf("not funny");
    }
    return 0;
}
```

Alternating Characters

```
#include <stdio.h>
```

```
#include <string.h>
#include <math.h>
#include <stdlib.h>

int main()
{
    int t;
    long i=0;
    unsigned int count=0;
    char * c;
    scanf("%d",&t);
    c=(char *)malloc(sizeof(char)*(100002));
    while(t--)
    {
        scanf("%s",c);
        for(i=0; c[i] != '\0'; i++)
        {
            if(c[i]==c[i+1])
            {
                count++;
            }
        }
        printf("%u\n",count);
        count=0;
    }

    return 0;
}
```

Beautiful Binary String

```
#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <limits.h>
#include <stdbool.h>

int main(){
    int n;
    scanf("%d",&n);
    char* B = (char *)malloc(10240 * sizeof(char));
    scanf("%s",B);
    int i=0,count=0;
    while(B[i]){
        if(B[i]=='0'&&B[i+1]=='1'&&B[i+2]=='0'){
        B[i+2]='1';
        count++;
        }
        i++;
    }
    printf("%d",count);
    return 0;
}
```

RANGE QUERIES

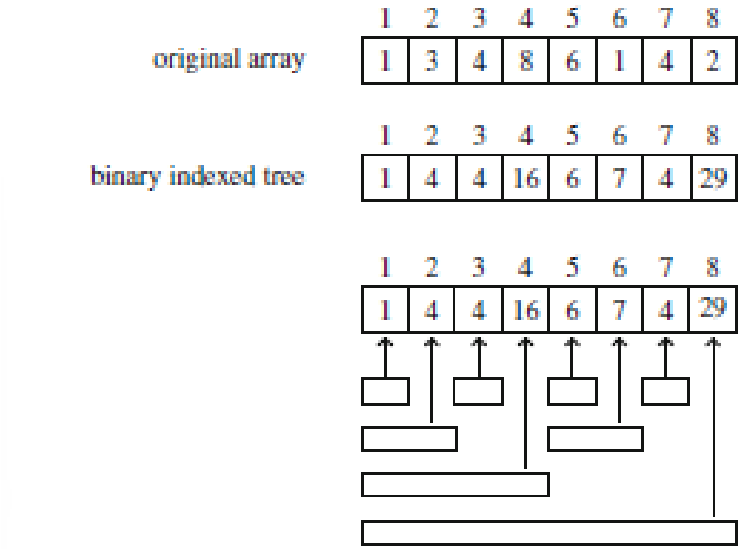
Fenwick Tree – Add a value

Given a Fenwick Tree (Binary indexed Tree) with the value of at node k calculated as

tree[k] = sum_q (k - p(k) + 1, k)

Where, p(k) = k&-k and denotes the largest power of two that divides k.

Create a function to increases the value at position k by x in the Fenwick Tree.




```
#include<stdio.h>
#include<math.h>


void display(int arr[], int n)
{
        int i;
        for (i=1;i<=n;i++)
        {
                printf("\t %d ",arr[i]);
```

```
        }
}

void add(int T[],int n, int k, int x) {
while (k <= n) {
T[k] += x;
k += k&-k;
}
display(T,n);
}
int main(){
        int n=8, k,x, T[100] = {0, 1, 4, 4, 16, 6, 7, 4, 29};
        printf("\n Fenwick Tree");
        display(T,n);
        printf("\n Enter the position and value to be increased ");
        scanf("%d%d",&k,&x);
          add(T,n, k, x);
        return 0;
}
```